
Chapter 14 Stacks and Queues

14.1 Stack/Queue Basics

- [Stack Concepts !\[\]\(c8dce68b26731c7aa5915072fc9d68dd_img.jpg\)](#)
- [Queue Concepts !\[\]\(76b3245de86167eba9fcdc9cc9f32aa4_img.jpg\)](#)

14.2 Common Stack/Queue Operations

- [Transferring between Stacks and Queues !\[\]\(f15d3c54be60b4fd0ce1da9fb3f67256_img.jpg\)](#)
- [Sum of a Queue !\[\]\(7bf135d42c40a6430c927b2fd03d7659_img.jpg\)](#)
- [Sum of a Stack !\[\]\(2bcc37677ea6b96900e4d746ad300082_img.jpg\)](#)

14.3 Complex Stack/Queue Operations

- [Removing Values from a Queue !\[\]\(b1b781be830eb908d845c527ab08d5f8_img.jpg\)](#)
- [Comparing Two Stacks for Similarity !\[\]\(2176a4ba510fa27404d783166e891577_img.jpg\)](#)

14.4 Case Study: Expression Evaluator

- [Splitting into Tokens !\[\]\(49aa2e1da5fe39294864e9598c593810_img.jpg\)](#)
- [The Evaluator !\[\]\(7d0a8d8b1031f74abe67b09fcf4a2322_img.jpg\)](#)

Introduction

*In **Chapter 11**, we saw that the Java Collections Framework incorporates the idea of an abstract data type (ADT) with interfaces defined for the most fundamental ADTs and a variety of implementations for each. In this chapter we are going to explore two of the most fundamental ADTs in computer science, called stacks and queues. They are so simple that they almost seem not worth studying. They are like the programming equivalent of drawers and shelves. Drawers and shelves are very simple and, therefore, sort of boring, and yet we find uses for them everywhere we turn.*

It is useful to study stacks and queues as a way to understand a minimal kind of data structure. We'll find, for example, that they are less powerful than the list structures we have been looking at. But we often find ourselves wanting to think in terms of the simplest possible solution to a problem, as in "Could you solve that with just a stack?" They are also part of the basic culture of computer science, so you will find it helpful to practice using these primitive

*structures and learning the terminology
associated with each.*

14.1 Stack/Queue Basics

Like a list, a stack or queue stores an ordered sequence of values. A minimal set of operations for such a structure would require at least:

- some way to put values into the structure (an “add” operation)
- a way to take values out (a “remove” operation)
- a way to test whether there is anything left in the structure (is the structure empty?)

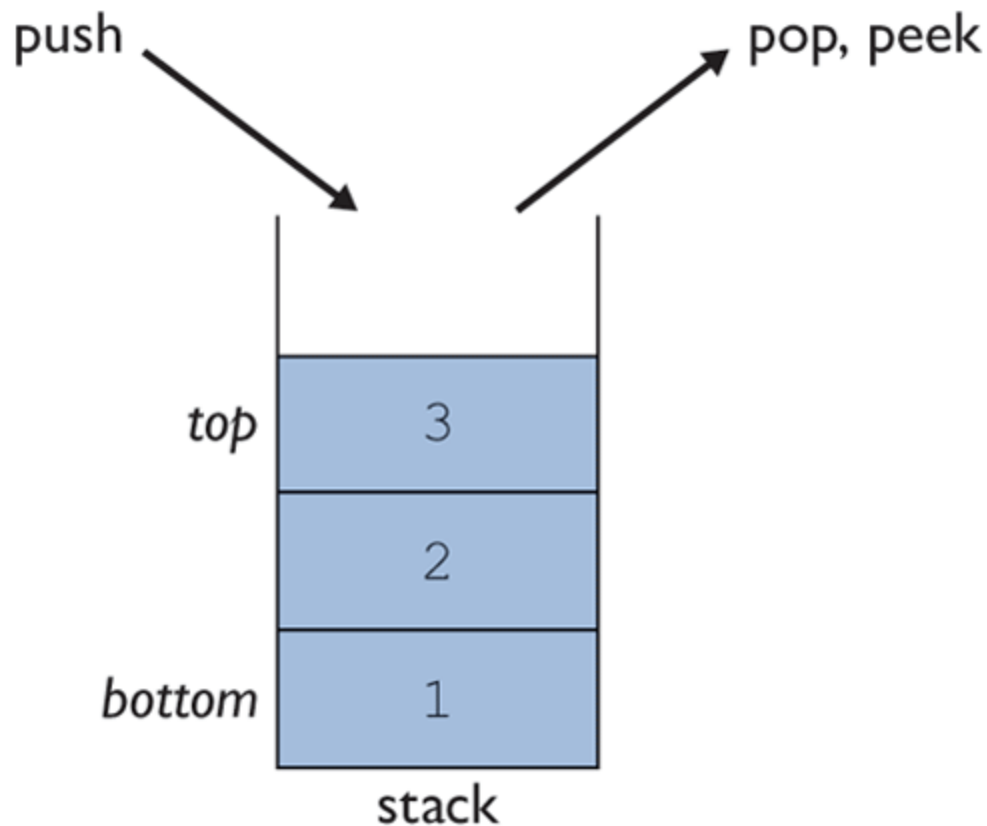
These three operations are the bare bones that you’d need for such a structure and in their purest form, stacks and queues have just these three operations. Java’s version of these also includes a `size` method that lets you ask for the number of elements in the structure and a `peek` method that lets you examine the next value to be removed without actually removing the value.

Stacks and queues are similar in that they each store a sequence of values in a particular order. Stacks are LIFO structures (Last-In, First-Out), meaning that the most recently inserted element is the one that can be accessed first. Queues are FIFO structures (First-In, First-Out), meaning that the first inserted element is the one that can be accessed first.

Stack Concepts

With a stack, all of the action occurs at one end of the structure that we refer to as the top of the stack, as shown in [Figure 14.1](#) . The adding operation is called a *push* and the removing operation is called a *pop*. We push values onto the top and we pop them off the top. A good analogy for stacks is to think of a cafeteria and how trays are stacked up for the customers. When you go to get a tray, you take the one on the top of the stack. You don't bother to try to get the one on the bottom, because you'd have to move a lot of trays to get to it. Similarly, if someone brings clean trays to add to the stack, they are added on the top rather than on the bottom. The result is that stacks tend to reverse things. Each new value goes to the top of the stack, and when we take them values back out, we draw from the top, so they come back out in reverse order.

Figure 14.1 A stack



There are also methods for testing whether the stack is empty, requesting the current size of the stack, and peeking at the next value to be removed. The basic operations for the `Stack` class are listed in [Table 14.1](#) .

Table 14.1 Useful Methods of the `Stack` Class

Stacks are generic structures like the other collections we have seen, so it is really `Stack<E>`, where `E` is filled in with some type. The `Stack` class is one of the oldest classes in the Java Collections Framework. It was included with the original release of Java. As a result, it was not designed as well as the later ADTs in the

framework. In particular, there is no separate interface that specifies the ADT. Instead, it extends a class called `Vector`, which is an early version of the `ArrayList` class. Because of this inheritance relationship, you can treat the stack like a list. For example, you can add and remove values in the middle.

Stack

An ordered collection that allows adding and removing the “top” element, providing Last-In, First-Out (LIFO) access.

Even though it is badly designed, we think it is important to use the built-in version of this class. We highly recommend that you restrict yourself to the stack-like methods of [Table 14.1](#) and not use other methods that would allow you to treat the stack like a list. The following `StackExample` is a complete program that shows basic manipulations of a stack.

```
1 // A program that demonstrates basic stack operations.
2
3 import java.util.*;
4
```

```

5 public class StackExample {
6     public static void main(String[] args) {
7         String[] data = {"to", "be", "or", "not", "to",
8         "be"};
9
10        Stack<String> s = new Stack<>();
11
12        for (String str : data) {
13            s.push(str);
14        }
15
16        System.out.println("stack = " + s);
17        System.out.println("size  = " + s.size());
18        System.out.println("peek  = " + s.peek());
19        while (!s.isEmpty()) {
20            System.out.print(s.pop() + " ");
21        }
22        System.out.println();
23    }
24 }

```

It produces the following output:

```

stack = [to, be, or, not, to, be]
size = 6
peek = be
be to not or be to

```

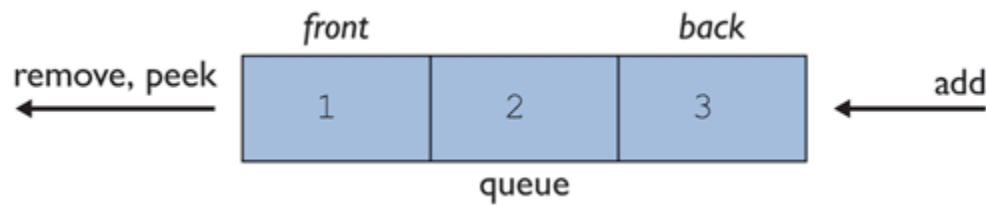
You can see from the first line of output that the `Stack` class has a `toString` method that formats the contents the same way a list is formatted. The first value listed inside brackets is the value at the bottom of the stack.

The second line of output uses the `size` method to report that there are 6 values in the stack. The third line of output uses the `peek` method to report that the top of the stack is `"be"`. Keep in mind that this peeking operation does not change the stack. The final line of output shows the sequence of values obtained by popping the stack until it becomes empty. Notice that the values come out in reverse order relative to the order in which they were added.

Queue Concepts

For a queue we manipulate both ends of the structure, adding new values at the back of the queue and removing values from the front, as shown in [Figure 14.2](#). Queues have the same basic operations that stacks have, but they have different names. The adding and removing operations are simply called `add` and `remove`, although sometimes we use the more formal terms *enqueueing* and *dequeueing* to describe these.

Figure 14.2 A queue



A good analogy for queues is to think about standing in line at the grocery store. As new people arrive, they are told to go to the back of the line. When the store is ready to help another customer, the person at the front of the line is helped. In fact, the British use the word “queue” the way Americans use the word “line,” telling people to “queue up” or to “go to the back of the queue.” [Table 14.2](#) includes the five fundamental operations we will use for manipulating queues.

Table 14.2 Useful Methods of the `Queue` Interface

Queues were added to the Java Collections Framework later and they were properly designed. There is a separate `Queue<E>` interface that describes the ADT and various implementations. We will use the `LinkedList<E>` implementation for all of the queues that we construct.

Queue

An ordered collection that allows adding the “back” element and removing the “front” element, providing First-In, First-Out (FIFO) access.

The following `QueueExample` is a simple client program that uses an array of `String` data to initialize and manipulate a queue. The program's output is similar to the earlier `StackExample` program, except for the order in which the elements are printed.

```
1  // A program that demonstrates basic queue operations.
2
3  import java.util.*;
4
5  public class QueueExample {
6      public static void main(String[] args) {
7          String[] data = {"to", "be", "or", "not", "to",
8                          "be"};
9
10         Queue<String> q = new LinkedList<>();
11
12         for (String str : data) {
13             q.add(str);
14         }
15
16         System.out.println("queue = " + q);
17         System.out.println("size  = " + q.size());
18         System.out.println("peek  = " + q.peek());
19         while (!q.isEmpty()) {
20             System.out.print(q.remove() + " ");
21         }
22     }
23 }
```

```
20         System.out.println();  
21     }  
22 }
```

The queue example produces the following output:

```
queue = [to, be, or, not, to, be]  
size = 6  
peek = to  
to be or not to be
```

In the first line of output you can see that the first value inside brackets is the value at the front of the queue. That is why the `peek` method returns `"to"`. And notice that the final line of output lists the values in the same order in which they were added to the structure.

14.2 Common Stack/Queue Operations



In this section we will become more familiar with stacks and queues by writing some client code for manipulating stacks and queues of `int` values. Recall that we have to use the `Integer` wrapper class to do so by constructing values of type `Stack<Integer>` and `Queue<Integer>`. Let's write some code that would add 10 random values between 0 and 99 to a queue using a `Random` object:

```
Queue<Integer> q = new LinkedList<>();  
Random r = new Random();  
for (int i = 0; i < 10; i++) {  
    q.add(r.nextInt(100));  
}
```

```
}  
System.out.println("q = " + q);
```

Notice that our variable is of type `Queue<Integer>` (using the interface). We only use the name of the implementation (`LinkedList`) when we are constructing a new object with the `new` keyword. All variables, parameters, and return types should be defined using the interface.

This produces output like the following:

```
q = [25, 19, 83, 0, 70, 52, 76, 33, 81, 54]
```

We can put this code for generating a queue with random values into a method. It's helpful to have a `size` parameter indicating how many values to add to the queue. The return type for the method should be `Queue<Integer>`:

```
public static Queue<Integer> makeRandomQueue(int size) {  
    Queue<Integer> q = new LinkedList<>();  
    Random r = new Random();  
    for (int i = 0; i < size; i++) {  
        q.add(r.nextInt(100));  
    }  
}
```

```
    return q;
}
```

We can change our client code to call this method and print the result:

```
Queue<Integer> q = makeRandomQueue(10);
System.out.println("q = " + q);
```

We can create a variation of `makeRandomQueue` called `makeRandomStack` for making a stack of random values. This is a fairly straightforward modification where we simply switch queue operations with stack operations:

```
public static Stack<Integer> makeRandomStack(int size) {
    Random r = new Random();
    Stack<Integer> s = new Stack<>();
    for (int i = 0; i < size; i++) {
        s.push(r.nextInt(100));
    }
    return s;
}
```

Transferring between Stacks and Queues

Consider the task of writing a method to transfer all values from a queue to a stack. We would need a loop that will remove values from the queue as long as there are more values left to remove. We can accomplish this with a `while` loop and the `isEmpty` and `remove` methods:

```
public static void queueToStack(Queue<Integer> q, Stack<Integer>
s) {
    while (!q.isEmpty()) {
        int n = q.remove();
        ...
    }
}
```

Notice that the parameters are of type `Queue<Integer>` and `Stack<Integer>`.

This code removes values from the queue, but to add them to the stack, we have to include a call on `push` inside the loop. The following is an incorrect attempt:

```
public static void queueToStack(Queue<Integer> q, Stack<Integer>
s) {
    while (!q.isEmpty()) {
        int n = q.remove();
        s.push(q.remove());
    }
}
```

The problem with this version is that it calls `remove` twice each time through the loop. It should be calling `push` with the value of `n`:

```
public static void queueToStack(Queue<Integer> q, Stack<Integer>
s) {
    while (!q.isEmpty()) {
        int n = q.remove();
        s.push(n);
    }
}
```

We could eliminate the line that involves `n` altogether and have just the second line of code:

```
s.push(q.remove());
```

The following client code reports what was in the stack and queue after calling `queueToStack`:

```
Queue<Integer> q = makeRandomQueue(10);  
Stack<Integer> s = new Stack<>();  
System.out.println("queue = " + q);  
System.out.println("stack = " + s);  
queueToStack(q, s);  
System.out.println("after queueToStack:");  
System.out.println("queue = " + q);  
System.out.println("stack = " + s);
```

It produces output like the following:

```
queue = [75, 76, 53, 82, 88, 77, 63, 28, 86, 7]  
stack = []  
after queueToStack:  
queue = []  
stack = [75, 76, 53, 82, 88, 77, 63, 28, 86, 7]
```

We can write a similar method `stackToQueue`:

```
public static void stackToQueue(Stack<Integer> s, Queue<Integer>  
q) {
```

```
while (!s.isEmpty()) {  
    int n = s.pop();  
    q.add(n);  
}  
}
```

Sum of a Queue

How would we write a method to find the sum of the values in a queue? It is a cumulative sum task, which involves initializing a `sum` variable to `0` outside the loop and then adding each value to the sum as we progress through the loop. A first attempt might look like this:

```
public static int sum(Queue<Integer> q) {  
    int sum = 0;  
    while (!q.isEmpty()) {  
        int n = q.remove();  
        sum += n;  
    }  
    return sum;  
}
```

If you call this version of the method and print the queue afterward, you'll find that the queue is empty. As a side effect of calculating the

sum, we are destroying the contents of the queue. This is generally not acceptable behavior.

So how do we examine each of the queue elements while preserving the state of the structure? You could ask the queue for an iterator or use a for-each loop to get each value from the structure, but we are interested in understanding the queue in its most basic form, so we aren't going to use either iterators or for-each loops when working with stacks and queues.

The only other way to find out what is in a queue is to remove each of the values. We just need to find a way to do that while restoring the queue to its original form when we are done. One solution is to use a second queue as auxiliary storage:

```
// Improved version that uses auxiliary queue for storage
// so that it does not destroy the queue passed in.
public static int sum(Queue<Integer> q) {
    int sum = 0;
    Queue<Integer> temp = new LinkedList<>();
    while (!q.isEmpty()) {
        int n = q.remove();
        sum += n;
        temp.add(n);
    }
    while (!temp.isEmpty()) { // restore the queue
        q.add(temp.remove());
    }
}
```

```
    return sum;
}
```

This approach works, but there is an easier way. Why not use the original queue itself? As we remove values to be processed, we can add them back into the queue at the end. Think of it as cycling through values where the value at the front goes to the back.

Of course, then the queue never becomes empty. So instead of a `while` loop looking for an empty queue, we would write a `for` loop using the size of the queue:

```
// Best version that re-adds elements to the queue passed in
// so that it does not destroy the contents of the queue.
public static int sum(Queue<Integer> q) {
    int sum = 0;
    for (int i = 0; i < q.size(); i++) {
        int n = q.remove();
        sum += n;
        q.add(n);
    }
    return sum;
}
```

Sum of a Stack

Now let's write a `sum` method for stacks:

```
public static int sum(Stack<Integer> s) {  
    ...  
}
```

This method can also be called `sum` because the two methods have different signatures. Remember that a signature of a method is its name plus its parameters. These are both called `sum` and they both have just a single parameter, but the parameter types are different, so this is okay.

So how do we write the `sum` method for stacks? We can start out by trying to simply substitute stack operations for queue operations:

```
public static int sum(Stack<Integer> s) {  
    int sum = 0;  
    for (int i = 0; i < s.size(); i++) {  
        int n = s.pop();  
        sum += n;  
        s.push(n);  
    }  
    return sum;  
}
```

Unfortunately, this code doesn't work. We'd see output like this when we test it:

```
stack = [42, 19, 78, 87, 14, 41, 57, 25, 96, 85]
sum = 850
```

The sum of these numbers is not 850. We're getting that sum because the loop pops the value 85 off the stack 10 different times and then pushes it back onto the top of the stack. With a queue, values go in at one end and come out the other end. But with a stack, all the action is at one end of the structure (the top). So this approach isn't going to work.

In fact, you can't solve this in a simple way with just a stack. You'd need something extra like an auxiliary structure. Consider how we could solve it if we were allowed to use one queue available (and only one queue; no other auxiliary structures) as temporary storage. Then we can put things into the queue as we take them out of the stack and after we have computed the sum, we can transfer things from the queue back to the stack using our `queueToStack` method. Here is a second attempt:

```
public static int sum(Stack<Integer> s) {
    int sum = 0;
    Queue<Integer> q = new LinkedList<>();
```

```

    for (int i = 0; i < s.size(); i++) {
        int n = s.pop();
        sum += n;
        q.add(n);
    }
    queueToStack(q, s);
    return sum;
}

```

This also doesn't work. Here is a sample execution:

```

initial stack = [32, 15, 54, 91, 47, 45, 88, 89, 13, 0]
sum = 235
after sum stack = [32, 15, 54, 91, 47, 0, 13, 89, 88, 45]

```

There are two problems here. Only half of the values were removed from the stack and those values now appear in reverse order. Why only half? We are using a `for` loop that compares a variable `i` against the size of the stack. The variable `i` is going up by 1 while the size is going down by 1 every time. The result is that halfway through the process, `i` is large enough relative to size to stop the loop. This is a case where we want a `while` loop instead of a `for` loop:

```
public static int sum(Stack<Integer> s) {  
    int sum = 0;  
    Queue<Integer> q = new LinkedList<>();  
    while (!s.isEmpty()) {  
        int n = s.pop();  
        sum += n;  
        q.add(n);  
    }  
    queueToStack(q, s);  
    return sum;  
}
```

Even this is not correct. It finds the right sum, but it ends up reversing the values in the stack. If we could use a stack instead of a queue as auxiliary storage, then this problem would go away. In many of our sample problems, we purposely restrict you to a particular kind of structure so that you can practice working within constraints. It becomes almost a brain teaser to think of how to solve the problem with a given structure. A stack would be more convenient, but that doesn't mean that you have to use a stack to solve the problem.

The problem is that by transferring the data from the stack into the queue and then back into the stack, we have reversed the order. The fix is to do it again so that it goes back to the original order. So we add two extra calls at the end of the method that move values from

the stack back into the queue and then from the queue back into the stack:

```
public static int sum(Stack<Integer> s) {  
    int sum = 0;  
    Queue<Integer> q = new LinkedList<>();  
    while (!s.isEmpty()) {  
        int n = s.pop();  
        sum += n;  
        q.add(n);  
    }  
    queueToStack(q, s);  
    stackToQueue(s, q);  
    queueToStack(q, s);  
    return sum;  
}
```

This is not the most efficient way to solve the problem, but it demonstrates that it can be done with a queue.

14.3 Complex Stack/Queue Operations



In this section, we will examine two harder stack/queue problems that allow us to explore some of the issues and common bugs that come up when manipulating these structures.

Removing Values from a Queue

Consider the task of removing all occurrences of a certain value from a queue of integers. For example, suppose that a variable called `q` stores a reference to the following queue:

```
[18, 4, 7, 42, 9, 33, -8, 0, 14, 42, 7, 42, 42, 19]
```

and we make the following call:

```
removeAll(q, 42);
```

We would want the queue to store the following values after the call:

```
[18, 4, 7, 9, 33, -8, 0, 14, 7, 19]
```

This seems like a fairly straightforward task, but it leads to a subtle bug if we aren't careful. This task would be easier to solve if we could use a second queue. Then we could copy the values we want to keep from the first queue to the second and then copy them back when we are done. But this problem can be solved without an auxiliary data structure.

We saw that we can cycle through the elements of a queue by repeatedly removing the front value and then adding it back at the end of the queue. For example, the following loop cycles through the values in the queue exactly once:

```
for (int i = 0; i < q.size(); i++) {  
    int n = q.remove();  
    q.add(n);  
}
```

As a first attempt, we could modify this loop so that it doesn't add the value back if it is the value we want to remove:

```
public static void removeAll(Queue<Integer> q, int value) {  
    for (int i = 0; i < q.size(); i++) {  
        int n = q.remove();  
        if (n != value) {  
            q.add(n);  
        }  
    }  
}
```

This version doesn't work. For the sample queue, if we remove 42, we end up with these values in the queue after the method finishes executing:

```
[42, 19, 18, 4, 7, 9, 33, -8, 0, 14, 7]
```

It's obvious that it didn't work because the queue now starts with a 42. Even worse, the order of the elements has been damaged. The original queue started with the value 18, which is now the third value in the queue. So it didn't finish the task and it has changed the order of the elements in the queue.

The problem is that our loop is using a counter to keep track of how many times to execute and it is comparing the counter against the size of the queue. But the size is changing. As we come across values to be removed, we skip them rather than adding them back into the queue, which makes the queue size go down by 1.

What happened in this specific case is that we came across two values of `42` that weren't added back in. That reduced the queue size by 2. As a result, the final two values from the original queue were never processed. That allowed a `42` to get past our filtering process, and it changed the order of the values in the queue because we didn't make a full pass through the data.

This is a subtle bug that can come up often when working with queues. You may think that you are making a complete cycle through the values when you are actually off by some amount that causes you to cycle either too little or too much.

In case the bug still isn't clear, let's look at a more extreme case. Suppose the queue consists entirely of six occurrences of the value `21`:

```
[21, 21, 21, 21, 21, 21]
```

If we call `removeAll` and pass this queue and the value `21`, the code should completely empty the queue. But the loop continues until the

counter `i` reaches the queue's `size`. The algorithm goes through the following iterations:

```
[21, 21, 21, 21, 21, 21], i = 0, size() = 6
[21, 21, 21, 21, 21], i = 1, size() = 5
[21, 21, 21, 21], i = 2, size() = 4
[21, 21, 21], i = 3, size() = 3
```

After three loop iterations and elements removed, the value of `i` has reached the `size` value of `3`, so the loop stops executing. The problem is that the queue is shrinking, so the `size` method will return different values on different iterations of the loop. Regardless of how many elements are removed, we always want the loop to repeat once for each element in the original queue. So we need to store the initial size of the queue in a variable and use that to control the loop:

```
public static void removeAll(Queue<Integer> q, int value) {
    int oldSize = q.size();
    for (int i = 0; i < oldSize; i++) {
        int n = q.remove();
        if (n != value) {
            q.add(n);
        }
    }
}
```

This version works properly.

Comparing Two Stacks for Similarity

Consider the task of comparing two stacks of integers to see if they have the same pattern of parity (odd versus even). In other words, we want to test whether two stacks have even numbers and odd numbers in the same positions. To be considered equivalent, we will also require that the stacks have the same size. For example, suppose variables called `s1` and `s2` store these values:

```
[1, 2, 3, 4, 5, 7, 9, 11]
[13, 4, 17, 8, 1, 91, 81, 71]
```

We expect the call of `sameParityPattern(s1, s2)` to return `true` because both stacks store values with an (odd, even, odd, even, ...) parity pattern. To solve this problem, we will need an auxiliary data structure. It would be most convenient to have two auxiliary stacks, but suppose that we have been restricted to one auxiliary stack. The basic form of our method will be to set up our auxiliary stack and examine values until we find a solution. We can also add some code to make sure that the two stacks are of the same size. We can begin our solution with this code:

```

public static boolean sameParityPattern(Stack<Integer> s1,
                                       Stack<Integer> s2) {
    if (s1.size() != s2.size()) {
        return false;
    } else {
        Stack<Integer> s3 = new Stack<>();
        ...
    }
}

```

Our task is to remove values from the two stacks and make sure they have the same parity. But we also have to be able to restore the two stacks to their original state. We have to use the third stack to store the values that we remove from the other two. If we find a pair of values that have different parity, then we would want to stop examining values. You might imagine writing code like the following:

```

while (!s1.isEmpty()) {
    int num1 = s1.pop();
    int num2 = s2.pop();
    if (num1 % 2 != num2 % 2) {
        return false;
    }
    s3.push(num1);
    s3.push(num2);
}

```

The preceding approach won't work. The code properly removes one value from each stack and stores the values into the third stack. But it returns from the method when it finds a difference. You have to be careful that you don't exit the method prematurely. Remember that we have to restore the stacks to their original state. So even though we know that we want to return `false`, we have to be careful to restore the stacks before we exit. We can accomplish this by introducing a `boolean` flag to keep track of whether we have seen a difference and stopping the loop when a difference has been encountered:

```
boolean same = true;
while (same && !s1.isEmpty()) {
    int num1 = s1.pop();
    int num2 = s2.pop();
    if (num1 % 2 != num2 % 2) {
        same = false;
    }
    s3.push(num1);
    s3.push(num2);
}
```

Now we need a loop that will restore the stacks to their original state. Our third stack has pairs of values that were originally in the other two stacks. We can repeatedly put each pair of values back to the appropriate stacks. In each pair, we first pushed a value from `s1` and

then pushed a value from `s2`, so you might think that the correct loop to write is:

```
while (!s3.isEmpty()) {  
    s1.push(s3.pop());  
    s2.push(s3.pop());  
}
```

The problem with this is that even though we pushed a value from `s1` and then a value from `s2`, they will come back out of the stack in the opposite order. So we need to reverse the order of the two lines inside our loop:

```
while (!s3.isEmpty()) {  
    s2.push(s3.pop());  
    s1.push(s3.pop());  
}
```

This completes our method other than the return statement. Once we have restored the stacks, we can use our `boolean` flag to report whether or not the stacks had the same pattern. The following is a complete solution to the method:

[illegible]

```
    if (s1.size() != s2.size()) {  
        return false;  
    } else {  
        Stack<Integer> s3 = new Stack<>();  
        boolean same = true;  
        while (same && !s1.isEmpty()) {  
            int num1 = s1.pop();  
            int num2 = s2.pop();  
            if (num1 % 2 != num2 % 2) {  
                same = false;  
            }  
            s3.push(num1);  
            s3.push(num2);  
        }  
        while (!s3.isEmpty()) {  
            s2.push(s3.pop());  
            s1.push(s3.pop());  
        }  
        return same;  
    }  
}
```

14.4 Case Study: Expression Evaluator

In [Chapter 12](#), we saw how to write a program that uses recursion to evaluate prefix expressions. We also saw that recursive solutions take advantage of the method call stack. But what if you didn't have recursion available to you? It turns out that many problems that are easily solved with recursion are also easily solved with a stack.

We are going to write a program that evaluates fully parenthesized arithmetic expressions using the standard operators for addition, subtraction, multiplication, and division, and a special operator for exponentiation ("[^]").

For example, if we wanted to evaluate this expression:

$$18.4 - \frac{2.3 \times 8.5}{19.5 + 2.7^{4.9}}$$

We would write it as a fully parenthesized expression as:

```
(18.4 - ((2.3 * 8.5) / (19.5 + (2.7 ^ 4.9))))
```

We have conventions that allow us to leave out many of the parentheses in this expression, but it is very challenging to process such expressions because then we have to handle precedence issues. We will keep it simple by always requiring parentheses for every operator.

In the [Chapter 12](#) case study, we assumed that spaces would be used to separate all of the individual tokens. That allowed us to use a `Scanner` for reading the tokens, but it can be very annoying to have to include spaces for each individual element. For example, the expression above would have to be rewritten as:

```
( 18.4 - ( ( 2.3 * 8.5 ) / ( 19.5 + ( 2.7 ^ 4.9 ) ) ) )
```

It turns out that separating a string like this into tokens leads to an interesting application for a queue. So we will write a more sophisticated version this time that allows you to have as little or as much spacing as you'd like.

We also should be careful to report most errors if the user leaves off parentheses or doesn't have them matched properly. This error checking is a nice complement to the other operations we will be performing to evaluate these expressions.

But we will avoid one thorny issue. We won't allow numbers to have a minus sign in front of them. So even though it makes sense to form an expression like this:

```
(-2.5/4.5)
```

We won't allow that minus sign in front of the `2.5`. Instead you'd have to form an expression like:

```
((0-2.5)/4.5)
```

Splitting into Tokens

We have seen that the `Scanner` class can be used to tokenize a string using whitespace, but we want to allow a user to leave out the whitespace and still have his or her input properly tokenized. We are going to build a supporting class called `StringSplitter` that will solve this fairly specialized task. It will behave somewhat like a `Scanner`, but it won't require whitespace to break apart the string.

We can use the standard method names from the `Scanner` class for getting a next token and asking if there is another token. We should also introduce a `peek` method that allows the client to ask about the next token without actually reading it. And we will need a constructor that takes the string to split as a parameter. So the public methods will be:

```
public StringSplitter(String line)
public boolean hasNext()
public String next()
public String peek()
```

This task is going to involve scanning through the letters of a string from beginning to end and returning to the client of the class the individual tokens that we find. Our class has to keep track of what has been read already and what it has left to read. It will also require a lot of peeking ahead to figure out how to separate the individual characters into tokens. For example, if the string is `"(2.3*4.8)"`, we will want to produce this sequence of tokens:

```
"(", "2.3", "*", "4.8", ")"
```

As we are reading a number like `2.3`, we will need to peek ahead to see what comes next so that we can recognize the end of the number.

It turns out that a queue is a great structure for solving this problem. It allows us to examine all of the characters from beginning to end and it keeps track of what we have already looked at and what we have left to look at. It also allows us to peek ahead.

The big difference between our splitter and a simple `Scanner` is that we need to recognize certain special characters as being tokens. If we encounter a parenthesis or one of the arithmetic operators, we have to make that a token whether or not it is surrounded by whitespace. It is best to include these special characters in a string constant for the class.

In terms of fields, since we are reading a character at a time, we will want a `Queue<Character>` to store the individual characters of the string and a field for storing the next token. Because we want to allow the client to peek ahead, we should have the field for the token always store the next token to be processed (that way we can allow the client to peek at it as much as he or she wants without actually reading anything).

We have to provide the client with a way to ask whether there is a next token to be processed. We could use an extra field for this, but we could also just set the token field to `null` when there are no tokens left to process.

Here is a basic outline of our class including our two fields, our special constant, and a constructor that adds the letters of the string to our queue of characters:

```
public class StringSplitter {  
    private Queue<Character> characters;  
    private String token;
```

```

    public static final String SPECIAL_CHARACTERS = "()+-*/^";

    public StringSplitter(String line) {
        characters = new LinkedList<>();
        for (int i = 0; i < line.length(); i++) {
            characters.add(line.charAt(i));
        }
        findNextToken();
        ...
    }

```

Notice that the constructor ends with a call on a method called `findNextToken`. The idea is that we want a private method that will process the queue and give an appropriate value to the field called `token`. Most of the work of writing this class is to write that method.

To write the `findNextToken` method, we have to consider all of the possible cases and make sure that we handle each one. We want to skip whitespace just as the `Scanner` does, so we will have to include code to do that. After skipping any leading whitespace, we would normally be ready to build up the next token. But we have to consider the case where we run out of characters, in which case there are no more tokens to produce. If we do have a token to produce, we'll have to remove characters one at a time from the queue and add them to the token that we are building up. The basic approach can be described with the following pseudocode:

```

skip whitespace.
if (nothing left) {
    token = null;
} else {
    initialize token to the next queue character.
    while (next queue character is part of this token) {
        token = token + (next character from the queue).
    }
}
}

```

To skip the leading whitespace, we can peek ahead in the queue to see if the next character is a whitespace character. We'll utilize a helpful static method from the `Character` wrapper class called `isWhitespace` that returns `true` if a given character is whitespace such as a space, tab, or line break. But we have to keep in mind we can't peek ahead when the queue is empty, so we have to include a special test for that as well:

```

while (!characters.isEmpty() &&
    Character.isWhitespace(characters.peek())) {
    characters.remove();
}
if (characters.isEmpty()) {
    token = null;
} else {

```

```
...  
}
```

Now we need to write the code for building up a token character by character. We can initialize the token to the next character in the queue by saying:

```
token = "" + characters.remove();
```

Then we run into the problem of knowing how many more characters to include in this token. We have a set of special characters that are supposed to be one-character tokens, so if the token is any of those, then we need to stop adding characters to the token. We can use our string constant and a call on `contains` to handle that special case:

```
if (!SPECIAL_CHARACTERS.contains(token)) {  
    ...  
}
```

Now we need a loop that will append characters to the current token until it finds something that isn't part of the token. We would want to stop if we came across a whitespace character. But we would also want to stop if we came across any of the special characters. And

we have an extra problem in that we might run out of characters completely if this is the last token to be processed.

The logic gets fairly complicated in this case, so it is helpful to introduce a `boolean` variable that keeps track of whether or not we are done:

```
boolean done = false;
while (!characters.isEmpty() && !done) {
    char ch = characters.peek();
    if (Character.isWhitespace(ch) ||
        SPECIAL_CHARACTERS.indexOf(ch) >= 0) {
        done = true;
    } else {
        token = token + characters.remove();
    }
}
```

This completes the private method for finding the next token. As mentioned earlier, the other parts of the class are fairly straightforward. The following is the complete class definition. Notice that the class has an extra private method called `checkToken` that throws a `NoSuchElementException` if the client calls the `peek` or `next` methods when there are no more tokens left.

```
1 // This class breaks up a string into a sequence of tokens
using
2 // both whitespace and a list of special characters that are
each
3 // considered tokens. The special characters in this case
are
4 // used to tokenize an arithmetic expression. For example,
the
5 // expression:
6 //      2*3.8/(4.95-7.8)
7 // would be tokenized as 2 * 3.8 / ( 4.95 - 7.8 ) even
though it
8 // has no whitespace to separate these tokens.
9
10 import java.util.*;
11
12 public class StringSplitter {
13     private Queue<Character> characters;
14     private String token;
15
16     public static final String SPECIAL_CHARACTERS = "()+-*/^";
17
18     public StringSplitter(String line) {
19         characters = new LinkedList<>();
20         for (int i = 0; i < line.length(); i++) {
21             characters.add(line.charAt(i));
22         }
```

```
23         findNextToken();
24     }
25
26     // post: Returns true if there is another token
27     public boolean hasNext() {
28         return token != null;
29     }
30
31     // pre : there is another token to return (throws
32     //      NoSuchElementException if not)
33     // post: returns and consumes the next token
34     public String next() {
35         checkToken();
36         String result = token;
37         findNextToken();
38         return result;
39     }
40
41     // pre : there is another token to return (throws
42     //      NoSuchElementException if not)
43     // post: returns the next token without consuming it
44     public String peek() {
45         checkToken();
46         return token;
47     }
48
49     // post: finds the next token, if any
50     private void findNextToken() {
```

```
51         while (!characters.isEmpty() &&
52             Character.isWhitespace(characters.peek())) {
53             characters.remove();
54         }
55         if (characters.isEmpty()) {
56             token = null;
57         } else {
58             token = "" + characters.remove();
59             if (!SPECIAL_CHARACTERS.contains(token)) {
60                 boolean done = false;
61                 while (!characters.isEmpty() && !done) {
62                     char ch = characters.peek();
63                     if (Character.isWhitespace(ch) ||
64                         SPECIAL_CHARACTERS.indexOf(ch) >= 0)
65                     {
66                         done = true;
67                     } else {
68                         token = token +
69                             characters.remove();
70                     }
71                 }
72             }
73
74             // post: throws an exception if there is no token left
75             private void checkToken() {
76                 if (!hasNext()) {
```

```
77         throw new NoSuchElementException();  
78     }  
79 }  
80 }
```

The Evaluator

Now that we have a support class that will allow us to read the tokens of a string, we can work on the code that will evaluate the tokens that we find in a fully parenthesized expression. We are going to implement a variation of a famous algorithm known as the shunting-yard algorithm that was invented by Edsger Dijkstra. It uses two stacks to save intermediate results. One stack stores numbers and the other stack stores symbols.

The basic idea is that we store values in the two stacks until we are ready to process them. As we see left parentheses and operators, we push them onto the symbol stack. As we see numbers, we push them onto the number stack. And when we see a right parenthesis, we know we have all of the information for a given sub-expression and we go ahead and evaluate it. We then push the result back onto the number stack.

Consider a simple case of evaluating `"(2+3)"`. [Table 14.3](#) shows how the initially empty stacks have elements added to them until we

encounter the right parenthesis, at which point we evaluate the sum and push the result onto the number stack.

Table 14.3 Evaluation of `" (2+3) "`

Notice that the overall value is the one and only value stored in the number stack when we are done and the symbol stack is empty when we are done. If there are other values left in either stack, then we know that we had an illegal expression.

This seems like a lot of work to do for a fairly simple computation. But remember that we can form complex sub-expressions that need to be evaluated as well. For example, what if the expression to evaluate had been `" ( (4/2) + (7-4) ) "`? This expression will have the same value because `(4/2)` evaluates to `2` and `(7-4)` evaluates to `3`. With the two stack approach, we can keep track of each part of this expression until we are ready to process it. [Table 14.4](#)  shows the evaluation of the more complex expression.

Table 14.4 Evaluation of `" ( (4/2) + (7-4) ) "`

To write the code, we just have to implement the algorithm. It is easiest if we assume the input has no errors, but it's better to recognize errors when we can. We can't really recover from an error and it can get complex to report the nature of each error. So let's strike a middle ground of recognizing as many errors as we can, but giving just a simple error message if we encounter a mistake.

We know that we want to process tokens until we either encounter an error or run out of tokens. It is helpful to introduce a `boolean` flag that keeps track of whether an error has been seen. We expect to reach a point where the symbol stack is empty and the number stack has exactly one value in it. In that case, we could report that one number as the overall result. The basic structure of our solution is:

```
StringSplitter data = new StringSplitter(line);
Stack<String> symbols = new Stack<>();
Stack<Double> values = new Stack<>();
boolean error = false;
while (!error && data.hasNext()) {
    ...
}
if (error || values.size() != 1 || !symbols.isEmpty()) {
    System.out.println("illegal expression");
} else {
    System.out.println(values.pop());
}
```

Our remaining task is to fill in the body of the `while` loop for processing tokens. Two of the cases are relatively simple. When we see a left parenthesis or operator, we push it on the symbol stack. When we see a number, we push it on the number stack. The hard part is when we see a right parenthesis. The body of our `while` loop will look like this:

```
String next = data.next();  
if (next.equals("(")) {  
    // process )  
} else if ("(+*/^".contains(next)) {  
    symbols.push(next);  
} else { // it should be a number  
    values.push(Double.parseDouble(next));  
}
```

Notice that in the final case we call the method `Double.parseDouble` to convert the token from a string into a `double`. As noted above, two out of three of these cases are simple. The third case is the hard one. How do we process a right parenthesis?

If an expression is legal, then we should have encountered a left parenthesis at the beginning of it and two numbers to work with and an operator to evaluate. So we expect that the symbol stack will have an operator on the top of the stack and a left parenthesis just below it. The two values should be on the number stack. In general we will remove the operator and left parenthesis, remove the two numbers, and then apply the operator. What makes this complicated is that there might be all sorts of errors. We have to be careful to check at every step that we have what we are supposed to have.

One thing we know is that the only values that are pushed onto the symbol stack are legal operators and left parentheses. We can use the stack sizes for several of our tests, but we have to make sure

that we have an operator and not a left parenthesis and that below it on the stack is a left parenthesis. The following code checks for these errors and evaluates one operator, pushing the result back onto the numbers stack:

```
if (symbols.size() < 2 || symbols.peek().equals("(")) {  
    error = true;  
} else {  
    String operator = symbols.pop();  
    if (!symbols.peek().equals("(")) {  
        error = true;  
    } else {  
        symbols.pop(); // to remove the "("  
        double op2 = values.pop();  
        double op1 = values.pop();  
        double value = evaluate(operator, op1, op2);  
        values.push(value);  
    }  
}
```

The code above involves a call on a method called `evaluate`. We wrote this method for the [Chapter 12](#) case study. It takes an operator and two operands and returns the result of applying that operator to the two operands. Putting all of this together, we end up with the following complete program:

```

1  // This program prompts for fully parenthesized arithmetic
2  // expressions and it evaluates each expression.  It uses two
3  // stacks to evaluate the expressions.
4
5  import java.util.*;
6
7  public class Evaluator {
8      public static void main(String[] args) {
9          System.out.println("This program evaluates fully");
10         System.out.println("parenthesized expressions with
the");
11         System.out.println("operators +, -, *, /, and ^");
12         System.out.println();
13         Scanner console = new Scanner(System.in);
14         System.out.print("expression (return to quit)? ");
15         String line = console.nextLine().trim();
16         while (line.length() > 0) {
17             evaluate(line);
18             System.out.print("expression (return to quit)?
");
19             line = console.nextLine().trim();
20         }
21     }
22
23     // pre : line contains a fully parenthesized expression
24     // post: prints the value of the expression or an error
25     //      message if the expression is not legal
26     public static void evaluate(String line) {

```

```
27         StringSplitter data = new StringSplitter(line);
28         Stack<String> symbols = new Stack<>();
29         Stack<Double> values = new Stack<>();
30         boolean error = false;
31         while (!error && data.hasNext()) {
32             String next = data.next();
33             if (next.equals("(")) {
34                 if (symbols.size() < 2 ||
35                     symbols.peek().equals("(")) {
36                     error = true;
37                 } else {
38                     String operator = symbols.pop();
39                     if (!symbols.peek().equals("(")) {
40                         error = true;
41                     } else {
42                         symbols.pop(); // to remove the "("
43                         double op2 = values.pop();
44                         double op1 = values.pop();
45                         double value = evaluate(operator,
46 op1, op2);
47                         values.push(value);
48                     }
49                 } else if ("(+-*/^".contains(next)) {
50                     symbols.push(next);
51                 } else { // it should be a number
52                     values.push(Double.parseDouble(next));
53                 }
```

```

54         }
55         if (error || values.size() != 1 ||
!symbols.isEmpty()) {
56             System.out.println("illegal expression");
57         } else {
58             System.out.println(values.pop());
59         }
60     }
61
62     // pre : operator is one of +, -, *, /, or ^
63     // post: returns the result of applying the given
operator to
64         //         the given operands
65     public static double evaluate(String operator, double
operand1,
66                                     double operand2) {
67         if (operator.equals("+")) {
68             return operand1 + operand2;
69         } else if (operator.equals("-")) {
70             return operand1 - operand2;
71         } else if (operator.equals("*")) {
72             return operand1 * operand2;
73         } else if (operator.equals("/")) {
74             return operand1 / operand2;
75         } else if (operator.equals("^")) {
76             return Math.pow(operand1, operand2);
77         } else {
78             throw new RuntimeException(

```

```
79         "illegal operator " + operator);  
80     }  
81 }  
82 }
```

Below is a sample log of execution:

```
This program evaluates fully  
parenthesized expressions with the  
operators +, -, *, /, and ^  
expression (return to quit)? (2+3)  
5.0  
expression (return to quit)? ((4-2)+(7-4))  
5.0  
expression (return to quit)? (2+3-4)  
illegal expression  
expression (return to quit)? (19.4-3.8))  
illegal expression  
expression (return to quit)? ((7.5/(2.3^7.2))-(9.4-3.8)) -  
5.581352490199907  
expression (return to quit)?
```

This program is fairly robust. The one bit of error checking it doesn't do is to handle illegal tokens. Any illegal tokens will be assumed to be numbers. The main token-processing loop includes a call on

`Double.parseDouble` that will throw a `NumberFormatException` if it encounters such a token. This could be fixed by adding a `try/catch` block that sets the error flag to true if the `NumberFormatException` is thrown.

Chapter Summary

A stack is a collection that allows you to add and remove elements from its top, providing “Last-In, First-Out” (LIFO) access.

The common operations of a stack include adding (“push”), removing (“pop”), testing whether the stack is empty, asking for the stack’s size, and “peeking” at the top element without removing it.

A queue is a collection that allows you to add elements to the back and remove elements from the front, providing “First-In, First-Out” (FIFO) access.

The common operations of a queue include adding (“enqueue”), removing (“dequeue”), testing whether the queue is empty, asking for the queue’s size, and “peeking” at the front element without removing it.

To process all elements of a stack, the collection must be emptied. If you want to examine the contents without damaging the collection, you must keep a backup and restore the data afterward.

To process all elements of a queue, you must either backup and restore the data or cycle values to the end of the queue as you process them.

The size of a stack or queue changes as its elements are processed and removed, so many algorithms to process these collections need to keep track of the collection's size separately to avoid common bugs and pitfalls.

Self-Check Problems

Section 14.1: Stack/Queue Basics

1. Which of the following statements about stacks and queues is true?
 - a. Stacks and queues can store only integers as their data.
 - b. A stack returns elements in the same order as they were added (first-in, first-out).
 - c. A queue's `remove` method removes and returns the element at the front of the queue.
 - d. Stacks and queues are similar to lists, but less efficient.
 - e. The `peek` method allows access to the element at the bottom of a stack.
2. What is a real-world example of data that could be modeled using a stack? Using a queue?
3. When you call `push` on a stack, where is the new element placed relative to the other elements in the stack? When you call `pop`, which element from the stack is returned?
4. When you call `add` on a queue, where is the new element placed relative to the other elements in the queue? When you call `remove`, which element from the queue is returned?
5. If you create a new empty stack and push the values 1, 2, and 3 in that order, and call `pop` on the stack once, what value will be returned?

6. If you create a new empty queue and add the values 1, 2, and 3 in that order, and call `remove` on the queue once, what value will be returned?
7. The following piece of code incorrectly attempts to declare a queue of integers. What is wrong with the code, and how would you fix it?

```
Queue<Integer> q = new Queue<>();
```

8. Write a piece of code that declares a stack of strings and fills it with the following contents, such that `"howdy"` is at the top of the stack and `"hello"` is at the bottom: `[hello, hi, goodbye, howdy]`. You can print the stack to verify its state.
9. Write a piece of code that declares a stack of integers and uses a loop to fill it with the multiples of 2 from 0 through 100 inclusive, such that `0` is at the top of the stack and `100` is at the bottom: `[100, 98, 96, ..., 4, 2, 0]`. You can print the stack to verify its state.
10. Write a piece of code that declares a queue of strings and fills it with the following contents, such that `"alpha"` is at the front of the queue and `"delta"` is at the back: `[alpha, beta, gamma, delta]`. You can print the queue to verify its state.

Section 14.2: Common Stack/Queue Operations

11. Stacks and queues do not have index-based methods such as `get` from `ArrayList`. How can you access elements in the middle of a stack or queue?
12. Stacks and queues have less functionality than other similar collections like lists and maps. Why are they still useful despite lacking functionality? What possible advantages are there of using a less powerful collection?
13. What is the output of the following code?

```
Stack<String> s = new Stack<>();  
Queue<String> q = new LinkedList<>();  
s.push("how");  
s.push("are");  
s.push("you");  
while (!s.isEmpty()) {  
    q.add(s.pop());  
}  
System.out.println(q);
```

14. What is the output of the following code?

```
Stack s = new Stack<>();  
s.push(7);
```

```
s.push(10);  
System.out.println(s.pop());  
System.out.println(s.peek());  
s.push(3);  
s.push(5);  
System.out.println(s.pop());  
System.out.println(s.isEmpty());  
System.out.println(s.size());  
System.out.println(s.peek());  
s.push(8);  
System.out.println(s.pop());  
System.out.println(s.pop());
```

15. What is the output of the following code?

```
Queue<Integer> q = new LinkedList<>();  
q.add(10);  
q.add(4);  
System.out.println(q.size());  
System.out.println(q.peek());  
q.add(6);  
System.out.println(q.remove());  
q.add(3);  
System.out.println(q.remove());  
System.out.println(q.peek());  
System.out.println(q.remove());  
q.add(7);  
System.out.println(q.peek());
```

16. Write the output produced when the following method is passed each of the following stacks:

```
public static void mystery1(Stack<Integer> s) {  
    Queue<Integer> q = new LinkedList<>();  
    while (!s.isEmpty()) {  
        int n = s.pop();  
        q.add(n);  
        q.add(n);  
    }  
    while (!q.isEmpty()) {  
        s.push(q.remove());  
    }  
    System.out.println(s);  
}
```

- a. [2, 6, 1]
- b. [42, -3, 4, 15, 9]
- c. [30, 20, 10, 60, 50, 40]

Section 14.3: Complex Stack/Queue Operations

17. Write the output produced when the following method is passed each of the following queues:

```
public static void mystery2(Queue<Integer> q) {  
    Stack<Integer> s = new Stack<>();  
    int size = q.size();  
    for (int i = 0; i < size; i++) {  
        int n = q.remove();  
        if (n % 2 == 0) {  
            s.push(n);  
        } else {  
            q.add(n);  
        }  
    }  
    System.out.println(q + " " + s);  
}
```

- a. [1, 2, 3, 4, 5, 6]
- b. [42, -3, 4, 15, 9, 71]
- c. [30, 20, 10, 60, 50, 40, 3, 0]

18. Write the output produced when the following method is passed each of the following queues:

```

public static void mystery3(Queue<Integer> q) {
    int size = q.size();
    for (int i = 0; i < size; i++) {
        int n = q.remove();
        if (n > 0) {
            q.add(-n);
        }
    }
    System.out.println(q);
}

```

- a. [1, -2, 3, -4, 5, -6]
- b. [42, -3, 4, -15, -9, 71]
- c. [-30, -20, 10, 60, 50, -40, -3, 0]

19. The following piece of code incorrectly attempts to find the largest value in a queue of integers. What is wrong with the code, and how would you fix it?

```

int largest = q.remove();
for (int i = 0; i < q.size(); i++) {
    largest = Math.max(largest, q.remove());
}

```

20. The following piece of code incorrectly attempts to compute the sum of all positive values in a queue of integers. What is

wrong with the code, and how would you fix it?

```
int sum = 0;
while (!q.isEmpty()) {
    if (q.remove() > 0) {
        sum += q.remove();
    }
}
```

21. The following piece of code incorrectly attempts to remove all even values from a stack of integers. What is wrong with the code, and how would you fix it?

```
while (!s.isEmpty()) {
    int n = s.pop();
    if (n % 2 != 0) {
        s.push(n); // put back in stack if odd
    }
}
```

22. Write a piece of code that prints the elements of a queue of integers, one per line. When your code is done running, the queue should still contain the same contents as it had at the start. In other words, don't destroy the queue as you print it. If you like, put your code into a method called `print` that accepts the queue as a parameter.
23. Write a piece of code that finds and prints the longest string in a stack of strings. For example, in the stack `[hello, hi,`

`goodbye, howdy]`, the longest string is `"goodbye"`. When your code is done running, the stack should still contain the same contents as it had at the start. In other words, if you destroy the stack as you examine it, restore its state afterward. If you like, put your code into a method called `printLongest` that accepts the stack as a parameter.

Exercises

Each problem will indicate what kind of structure to use as auxiliary storage. You should not use any other auxiliary data structures to solve the problems, although you can create as many simple variables as you'd like.

It is the authors' intent that you use stacks/queues in stack/queue-like ways only when solving these problems. For example, you should not call index-based methods such as `get`, `search`, or `set` (or use a for-each loop) on a stack/queue. You may call only `add`, `remove`, `push`, `pop`, `peek`, `isEmpty`, and `size`. It is also possible to solve all of the exercises without using `peek` if you want an extra challenge.

For problems that accept a stack or queue as a parameter, unless otherwise specified, you should make sure that your method does not damage the state of the parameter. That is, if you modify the parameter stack or queue's elements in your method, you should restore the parameter collection to its original state before your method returns.

Some of these problems have elegant recursive solutions, but the authors' intent is generally that you should not solve these problems

recursively, because recursion can circumvent some of the tricky stack/queue manipulation that you are supposed to practice.

1. Write a method called `splitStack` that accepts a stack of integers as a parameter and rearranges its elements so that all the negatives appear on the bottom of the stack and all the nonnegatives appear on the top. If after this method is called you were to pop numbers off the stack, you would first get all the nonnegative numbers and then get all the negative numbers. It does not matter what order the numbers appear in as long as all the negatives appear lower in the stack than all the nonnegatives. For example, if the stack stores `[3, -5, 1, 2, -4]`, an acceptable result from your method would be `[-5, -4, 3, 1, 2]`. Use a single queue as auxiliary storage.
2. Write a method called `stutter` that accepts a stack of integers as a parameter and replaces every value in the stack with two occurrences of that value. Preserve the original relative order. For example, if the stack stores `[3, 7, 1, 14, 9]`, your method should change it to store `[3, 3, 7, 7, 1, 1, 14, 14, 9, 9]`. Use a single queue as auxiliary storage.
3. Write a method called `copyStack` that accepts a stack of integers as a parameter and returns a copy of the original stack (i.e., a new stack with the same values as the original, stored in the same order as the original). Your method should create the new stack and fill it up with the same values that are stored in the original stack. When your method is done

executing, the original stack must be restored to its original state. Use one queue as auxiliary storage.

4. Write a method called `collapse` that accepts a stack of integers as a parameter and that collapses it by replacing each successive pair of integers with the sum of the pair. For example, if the stack stores `[7, 2, 8, 9, 4, 11, 7, 1, 42]`, the first pair should be collapsed into $9(7 + 2)$, the second pair should be collapsed into $17(8 + 9)$, and so on. If the stack stores an odd number of elements, such as the `42` at the end of our example stack, the final element is not collapsed. So for this stack your method would yield `[9, 17, 15, 8, 42]`. Use one queue as auxiliary storage.
5. Write a method called `equals` that accepts two stacks of integers as parameters and returns `true` if the two stacks store exactly the same sequence of integer values in the same order. Your method must restore the two stacks to their original state before returning. Use one stack as auxiliary storage.
6. Write a method called `rearrange` that accepts a queue of integers as a parameter and rearranges the order of the values so that all of the even values appear before the odd values and that otherwise preserves the original order of the queue. For example, if the queue stores `[3, 5, 4, 17, 6, 83, 1, 84, 16, 37]`, your method should rearrange it to store `[4, 6, 84, 16, 3, 5, 17, 83, 1, 37]`. Notice that all of the evens appear at the front followed by the odds and that the relative

order of the evens and odds is the same as in the original. Use one stack as auxiliary storage.

7. Write a method called `reverseHalf` that accepts a queue of integers as a parameter and reverses the order of all the elements in odd-numbered positions (position 1, 3, 5, etc.), assuming that the first value in the queue has position 0. For example, if the queue stores `[1, 8, 7, 2, 9, 18, 12, 0]`, your method should change it to store `[1, 0, 7, 18, 9, 2, 12, 8]`. Notice that numbers in even positions (positions 0, 2, 4, 6) have not moved. That subsequence of integers is still (1, 7, 9, 12). But notice that the numbers in odd positions (positions 1, 3, 5, 7) are now in reverse order relative to the original. In other words, the original subsequence (8, 2, 18, 0) has become (0, 18, 2, 8). Use a single stack as auxiliary storage.
8. Write a method called `isPalindrome` that accepts a queue of integers as a parameter and returns `true` if the numbers in the queue are the same in reverse order. For example, if the queue stores `[3, 8, 17, 9, 17, 8, 3]`, your method should return `true` because this sequence is the same in reverse order. If the queue stores `[3, 17, 9, 4, 17, 3]`, your method would return `false` because this sequence is not the same in reverse order (the 9 and 4 in the middle don't match). The empty queue should be considered a palindrome. Your method must restore the parameter queue to its original state before returning. Use one stack as auxiliary storage.
9. Write a method called `switchPairs` that accepts a stack of integers as a parameter and swaps neighboring pairs of

numbers starting at the bottom of the stack. For example, if the stack initially stores `[1, 2, 8, 6, -1, 15, 7]`, your method should swap the first pair (1, 2), the second pair (8, 6), the third pair (-1, 15), and so on. If the stack contains an odd number of elements, the element at the top should remain unmodified. So the final state of the stack would be `[2, 1, 6, 8, 15, -1, 7]`. Use one queue as auxiliary storage.

10. Write a method called `isConsecutive` that accepts a stack of integers as a parameter and that returns `true` if the stack contains a sequence of consecutive integers starting from the bottom of the stack. Consecutive integers are integers that come one after the other, as in 3, 4, 5, etc. If the stack stores `[5, 6, 7, 8, 9, 10]`, your method should return `true`. If the stack had instead contained `[7, 8, 9, 10, 12]`, your method should return `false` because the numbers 10 and 12 are not consecutive. Notice that we look at the numbers starting at the bottom of the stack. Any stack with fewer than two values should be considered to be a list of consecutive integers. Your method must restore the parameter stack to its original state before returning. Use one queue as auxiliary storage.
11. Write a method called `reorder` that accepts a queue of integers as a parameter and that puts the integers into sorted (nondecreasing) order, assuming that the queue is already sorted by absolute value. For example, if the queue stores `[1, 2, -2, 4, -5, 8, -8, 12, -15]`, notice that the values appear in sorted order if you ignore the sign of the numbers. Your method should reorder the values so that the queue stores

`[-15, -8, -5, -2, 1, 2, 4, 8, 12]`. Use one stack as auxiliary storage.

12. Write a method called `shift` that accepts a stack of integers and an integer n as parameters and that shifts n values from the bottom of the stack to the top of the stack. For example, if the stack named `s` stores `[1, 2, 3, 4, 5, 6, 7, 8]`, and we make the call `shift(s, 3)`; your method should shift the three values at the bottom of the stack to the top of the stack and leave the other values in the same order, producing `[4, 5, 6, 7, 8, 3, 2, 1]`. Notice that the value that was at the bottom of the stack is now at the top, the value that was second from the bottom is now second from the top, the value that was third from the bottom is now third from the top, and that the five values not involved in the shift are now at the bottom of the stack in their original order. Use one queue as auxiliary storage. You may assume that the parameter n is ≥ 0 and not larger than the number of elements in the stack.
13. Write a method called `expunge` that accepts a stack of integers as a parameter and makes sure that the stack's elements are in nondecreasing order from top to bottom, by removing from the stack any element that is smaller than any element(s) on top of it. For example, if the stack stores `[4, 20, 15, 15, 8, 5, 7, 12, 3, 10, 5, 1]`, the element values `3, 7, 5, 8,` and `4` should be removed because each has an element above it with a larger value. So your method should change the stack to store `[20, 15, 15, 12, 10, 5, 1]`. Notice that now the elements are in nondecreasing order from top to bottom. If the

stack is empty or has just one element, nothing changes. Use one queue or stack (but not both) as auxiliary storage.

14. Write a method called `reverseFirstK` that accepts an integer k and a queue of integers as parameters and reverses the order of the first k elements of the queue, leaving the other elements in the same relative order. For example, if a queue named `q` stores `[10, 20, 30, 40, 50, 60, 70, 80, 90]`, the call of `reverseFirstK(4, q);` should change the queue to store `[40, 30, 20, 10, 50, 60, 70, 80, 90]`. If k is 0 or negative, no change should be made. If the queue does not contain at least k elements, your method should throw an `IllegalArgumentException`. Use one queue or stack (but not both) as auxiliary storage.
15. Write a method called `isSorted` that accepts a stack of integers as a parameter and returns `true` if the elements in the stack occur in ascending (nondecreasing) order from top to bottom. That is, the smallest element should be on top, growing larger toward the bottom. For example, if the stack stores `[20, 20, 17, 11, 8, 8, 3, 2]`, your method should return `true`. An empty or one-element stack is considered to be sorted. Your method must restore the parameter stack to its original state before returning. Use one queue or stack (but not both) as auxiliary storage.
16. Write a method called `mirror` that accepts a stack of integers as a parameter and replaces the stack contents with itself plus a mirrored version of itself (the same elements in the opposite order). For example, if the stack stores `[10, 53, 19,`

24], your method should change it to store [10, 53, 19, 24, 24, 19, 53, 10]. If passed an empty stack, your result should be an empty stack. Use one stack or one queue (but not both) as auxiliary storage to solve this problem.

17. Write a method called `compressDuplicates` that accepts a stack of integers as a parameter and that replaces each sequence of duplicates with a pair of values: a count of the number of duplicates, followed by the actual duplicated number. For example, if the stack stores [2, 2, 2, 2, 2, -4, -4, -4, 82, 6, 6, 6, 6, 17, 17], your method should change it to store [5, 2, 3, -4, 1, 82, 4, 6, 2, 17]. This new stack indicates that the original had 5 occurrences of 2 at the bottom of the stack followed by 3 occurrences of -4 followed by 1 occurrence of 82, and so on. If the stack is empty, your method should not change it. Use one queue as auxiliary storage.
18. Write a method called `mirrorHalves` that accepts a queue of integers as a parameter and replaces each half of that queue with itself plus a mirrored version of itself (the same elements in the opposite order). For example, if the queue stores [10, 50, 19, 54, 30, 67], your method should change it to store [10, 50, 19, 19, 50, 10, 54, 30, 67, 67, 30, 54]. If your method is passed an empty queue, the result should be an empty queue. If your method is passed a queue whose size is not even, throw an `IllegalArgumentException`. Use one stack or one queue (but not both) as auxiliary storage.

19. Write a method called `removeMin` that accepts a stack of integers as a parameter and removes and returns the smallest value from the stack. For example, if the stack stores `[2, 8, 3, 19, 2, 3, 2, 7, 12, -8, 4]`, your method should remove and return `-8`, leaving the stack storing `[2, 8, 3, 19, 2, 3, 2, 7, 12, 4]`. If the minimum value appears more than once, all occurrences of it should be removed. For example, given the same stack, if we again call `removeMin` on it, the method would return `2` and leave the stack storing `[8, 3, 19, 3, 7, 12, 4]`. Use one queue as auxiliary storage.
20. Write a method called `interleave` that accepts a queue of integers as a parameter and rearranges the elements by alternating the elements from the first half of the queue with those from the second half of the queue. For example, if the queue stores `[2, 8, -5, 19, 7, 3, 24, 42]`, your method should change it to store `[2, 7, 8, 3, -5, 24, 19, 42]`. To understand the result, consider the two halves of this queue. The first half is `[2, 8, -5, 19]` and the second half is `[7, 3, 24, 42]`. These are combined in an alternating fashion to form a sequence of pairs: the first values from each half (`2` and `7`), then the second values from each half (`8` and `3`), and so on. Your method should throw an `IllegalArgumentException` if the queue does not have an even size. Use one stack as auxiliary storage.
21. Write a method called `maxToTop` that takes a stack of integers as a parameter and moves the largest value in the stack to

the top of the stack, leaving all other values in their original order. You may assume that the stack does not contain any duplicates. For example, if a stack `s` stores `[27, 5, 42, -11, 0, 19]`, the call of `maxToTop(s)` should change it to store `[27, 5, -11, 0, 19, 42]`. If the stack is empty, your method should not change it. Use one queue as auxiliary storage.

Programming Projects

1. Write a Primes program that finds prime numbers using the Sieve of Eratosthenes, an algorithm devised by a Greek mathematician of the same name who lived in the third century BC. The algorithm finds all prime numbers up to some maximum value n , as described by the following pseudocode:

```
create a queue of numbers to process.  
fill the queue with the integers 2 through  $n$  inclusive.  
create an empty result queue to store primes.  
  
repeat the following steps:  
    obtain the next prime  $p$  by removing the first value  
    from the queue of numbers.  
    put  $p$  into the result queue of primes.  
    loop through the queue of numbers,  
    eliminating all numbers that are divisible by  $p$ .  
while ( $p$  is less than the square root of  $n$ ).  
  
all remaining values in the numbers queue are prime,  
so transfer them to the result primes queue.
```

Several web sites have nice descriptions and animations of this algorithm in action; consider searching for “Sieve of Eratosthenes” in your web browser.

2. Write an HTML Validator program that reads files of HTML data and uses stacks and queues to verify whether the tags in the file are properly matched. A tag consists of a named element between less-than, `<`, and greater-than, `>`, symbols. Many tags apply to a range of text, in which case a pair of tags is used: an opening tag indicating the start of the range and a closing tag with a slash indicating the end of the range. For example, you can make some text bold `<b>` like this `</b>`. Tags can be nested to combine effects, `<b><i>` bold italic `</i>` `</b>`. Some tags, such as the `br` tag for inserting a line break or `img` for inserting an image, do not cover a range of text and are considered to be “self-closing.” Self-closing tags do not need a closing tag; for a line break, only a tag of `<br>` is needed. Some web developers write self-closing tags with an optional `/` before the `>`, such as `<br/>`. The following HTML file has some errors: the `<title>` tag is not closed; the `</head>` tag appears twice; an extraneous `</br>` tag appears; and the `<body>` tag is not properly closed.

```
<!DOCTYPE html>

<html>

  <!-- This is a comment -->

  <head>

    <title>Turtles are cool

  </head>

  </head>

  </br>

  <body>
```

```
<body>
  <p>Turtles swim in the <a href="ocean.com">ocean</a>.
</p>
</br>
  <p>Some turtles are over 100 years old.
  Here is a picture of a turtle:
  </p>
</html>
```

3. Modify the expression evaluator case study program from this chapter to make a new program that accepts as input a fully parenthesized infix expression and returns a string representing an equivalent postfix expression. Postfix expressions are ones where each operator follows its two operands, such as `1 2 +` rather than `1 + 2`. Postfix expressions are elegant in that they do not need parentheses. For example, the given infix expression:

```
( 9 + ( 8 * 7 - ( 6 / 5 ^ 4 ) * 3 ) * 2 ) )
```

is equivalent to the following postfix expression:

```
9 8 7 * 6 5 4 ^ / 3 * - 2 * +
```

Your algorithm should read the expression token by token, using a stack to store operators. (You don't need the values stack from the original case study anymore.) Instead, each

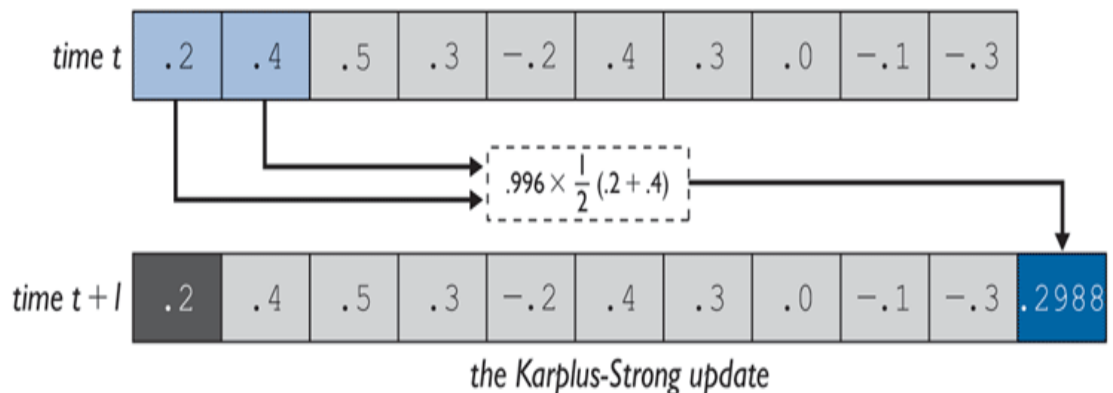
time a number is encountered, append it to a string that you are building up. Each time a right parenthesis is encountered, pop the stack to get an operator and append it to the string you are building up. Leave in place the rest of the code to preserve the error checking. Return the string `" illegal expression "` if an error is encountered.

4. Write a Maze Explorer program that uses stacks and queues to implement an algorithm to escape from a maze. The overall pseudocode of the algorithm is the following. The algorithm can be implemented using a stack or a queue. What are the pros and cons of each?

```
create an empty stack of locations to explore.
push the start location onto the stack.
|
while (stack is not empty) {
    pop a location L from the stack.
    if we have we pulled L from the stack before:
        no need to explore it again, so skip L.
    if L is the end location:
        the end was reachable!
    else, L is a new reachable non-finish location, so
    explore it:
        add all non-wall adjacent maze locations to the
        stack.
        record the fact that we have explored L.
}
if the stack is empty, the finish is unreachable.
```

5. Write a Guitar Hero program that uses a queue to simulate the creation of guitar notes. When a guitar string is plucked, it vibrates to generate sound. The sound starts from the note's initial note pitch or frequency, then it undergoes a wave-like oscillation and gradually fades in volume over time. The oscillation and fading of notes can be computed using an algorithm called Karplus-Strong. The algorithm represents sound as time slices called samples (44,100 samples per second, in this case). You can compute samples that sound similar to the vibrations of a real guitar as displacements from the guitar string's original frequency. First create a queue of random displacements between $-\frac{1}{2}$ and $\frac{1}{2}$. The length of the queue should be the sampling rate, 44,100, divided by the note's frequency. Then repeatedly remove the first queue element, average it with the next front element, slightly fade the volume by multiplying it by 0.996, then add the result back into the queue as shown in [Figure 14.3](#).

Figure 14.3 Karplus-Strong algorithm samples



This program requires support code to send output to your computer's sound card. The code is provided on our web site at <http://buildingjavaprograms.com/>.